

WMS AI Analysis — Requirements Document

Version 1.0 · 2026-09-12 · Source: WebView app v10.0.0 (aianalysis/) · Target: React AI assistant

Complete specification of the AI Analysis chatbot as built, so the same feature set can be implemented in the React-based assistant. Everything on the database / ORDS side is **shared infrastructure** — the React app reuses it unchanged; only the UI layer and the Claude transport are reimplemented.

1 Overview

#	Capability	Mechanism
1	Natural-language questions → SQL answers	Guarded SELECT-only gateway (ai/executequery)
2	Oracle Fusion REST calls (reads + approved writes)	Direct HTTPS with Basic auth; writes need approval card
3	Interactive selection grids (tick rows → action)	grid action + GRID_ACTION feedback
4	WMS write operations via interactive forms	Write-API catalog + api_form action + confirm + audit log
5	Saved re-runnable reports with parameters	WMS_AI_REPORTS tables + ai/reports/* endpoints
6	Email results (Office 365)	Approval card → SMTP (desktop app sends)
7	Approved DDL/DML from chat	db_write action → ai/executewrite
8	Scheduled background jobs (repeat-until-done)	schedule_job action → DBMS_SCHEDULER via ai/jobs/*
9	PROD/TEST instance awareness everywhere	Instance chip + [CURRENT_INSTANCE] context line
10	Dual engine: Claude CLI (subscription) or Claude API (key)	Per-client setting; same protocol both ways

Core design rule: *all reads are SQL; webservices are used only for writes (POST/PUT/DELETE).* Every write — Fusion, DB, email, job, WMS API — pauses for explicit user approval before executing, and API runs are audit-logged.

2 Architecture

```

User -> Chat UI -> Claude engine (CLI subprocess OR api.anthropic.com)
                        | replies with EXACTLY ONE JSON action
                        v
                        Action dispatcher (loop, max 5 tool rounds)
sql                -> POST ai/executequery          -> SQL_RESULT: fed back
fusion GET         -> Fusion REST (basic auth)      -> FUSION_RESULT:
fusion write       -> PAUSE -> approval card -> resume
db_write           -> PAUSE -> card -> ai/executewrite -> DB_WRITE_RESULT:
schedule_job       -> PAUSE -> card -> ai/jobs/create  -> JOB_RESULT:
email              -> PAUSE -> card -> SMTP            -> EMAIL_RESULT:
save_report        -> POST ai/reports/save          -> REPORT_SAVE_RESULT:
api_form           -> END TURN -> render form -> user confirms ->
                        run API + log -> NEW TURN "API_RESULT: {...}"
grid / answer      -> END TURN -> render
  
```

- The model is instructed (system prompt) to reply with **exactly one JSON object** per turn.
- Tool results are fed back as a user message prefixed with a marker (SQL_RESULT:, FUSION_RESULT:, API_RESULT:, ...) and the loop continues until an answer / grid / api_form ends the turn. Budget: 5 tool rounds; one retry on malformed JSON.
- Approval-gated actions **end the server loop** and return a pending* object; the UI renders a card; the user's decision resumes the conversation (CLI: --resume <sessionId>; API mode: replay the saved conversation array).

3 Database objects

All in schema WKSP_GRAYSAPP; scripts in apex_sql/, run in numbered order. Already deployed for the WebView app — the React app reuses them.

3.1 Script 35 — query gateway foundations

Object	Type	Purpose
WMS_AI_OBJECT_ACL	table	Controls which tables/views the AI may see and query. Empty = everything visible; a row with N always hides; any Y row switches to whitelist mode. AI-internal tables are seeded N.
WMS_AI_QUERY_LOG	table	One row per executed AI query (sql, rowcount, elapsed, user, error).
WMS_AI_LOG_QUERY	procedure	Autonomous-transaction logger used by the executor.
COMMENT ON pass	—	Table/column comments on the 12 core wms_ tables; served to the model as schema documentation.

3.2 Scripts 35c + 36 — SQL executor, reports

Object	Type	Purpose
WMS_AI_EXECUTE_SQL	procedure	Shared guarded executor: strips comments, enforces single SELECT/WITH statement, keyword ban (INSERT/UPDATE/DELETE/DDL/PLSQL...), wraps with FETCH FIRST maxRows+1, describes/fetches via DBMS_SQL (numbers, dates, timestamps), returns {success, columns[], rows[][], rowCount, truncated} JSON, logs via WMS_AI_LOG_QUERY.
WMS_AI_EXECUTE_QUERY	procedure	Wrapper: parses :body_text {sql, maxRows, appUser} and calls the executor.
WMS_AI_REPORTS	table	Saved reports: report_id (identity), name, category, description, sql_text CLOB, created_by, created_date, last-run info.
WMS_AI_REPORT_PARAMS	table	Per-report parameters: param_name (bind, e.g. P_TRIP_DATE), label, param_type (TEXT/NUMBER/DATE), required Y/N, default value, display order.
WMS_AI_SAVE_REPORT	procedure	Parses the save payload, inserts report + params.
WMS_AI_RUN_REPORT	procedure	Substitutes param values into the saved SQL. DATE params: first unwraps any TO_DATE(:PARAM,'fmt') already in the SQL (avoids double TO_DATE), then substitutes a typed literal; executes via the shared executor.

3.3 Script 37 — approved DB writes

Object	Type	Purpose
WMS_AI_EXECUTE_WRITE	procedure	Executes ONE statement whose first verb is in: CREATE, ALTER, DROP, INSERT, UPDATE, DELETE, MERGE, COMMENT, TRUNCATE. Bans GRANT/REVOKE, anonymous PL/SQL, CALL, DBMS_/UTL_ packages, ALTER SESSION SYSTEM USER DATABASE. Returns {success, verb, rowsAffected} or {success:false, error}.

3.4 Script 38 — settings + scheduler (DBMS jobs)

Object	Type	Purpose
WMS_AI_SETTINGS	table	Key/value settings needed INSIDE the DB. Seeded keys: FUSION_USERNAME, FUSION_PASSWORD (job REST steps), FUSION_USERNAME_TEST / FUSION_PASSWORD_TEST (optional TEST pair, fallback to the main pair when NULL), FUSION_INSTANCE (default PROD/TEST for jobs; resolves #FUSION_BASE#).
WMS_AI_JOBS	table	Job header: name, description, steps_json CLOB ({"steps":[...]}), schedule_type ONCE/RECURRING/REPEAT_UNTIL_DONE, start_at, interval_minutes (min 2), completion_sql, max_runs (≤200), until_date (≤ +30 d), status SCHEDULED/RUNNING/COMPLETED/FAILED/CANCELLED/EXPIRED, instance, created_by / created_machine / created_date (audit), runs_count, last_run_at, last_error.
WMS_AI_JOB_RUNS	table	One row per run with full log_text CLOB (step-by-step log).

Object	Type	Purpose
WMS_AI_JOB_RUNNER	procedure	Executed by DBMS_SCHEDULER. Enforces caps (-> EXPIRED), reads Fusion creds + instance from settings, exposes #FUSION_BASE#, runs steps in order: sql steps = guarded SELECT COUNT(*); rest steps = APEX_WEB_SERVICE.MAKE_REST_REQUEST with host whitelist (own ORDS + efmh + efmh-test), optional auth:"fusion" basic auth, HTTP >=400 raises, extract captures JSON values (APEX_JSON paths, arrays 1-BASED) into #VAR# substitutions for later steps. REPEAT_UNTIL_DONE: when completion_sql returns 0 rows -> COMPLETED + self-disable.
WMS_AI_JOB_CREATE / _CANCEL / _RUNNOW	procedures	Create the DBMS_SCHEDULER job (AI_JOB_<id>, PLSQL_BLOCK, FREQ=MINUTELY, end_date, auto_drop for ONCE); drop-with-force + mark CANCELLED; RUN_JOB(use_current_session=>FALSE).

3.5 Script 39 — API audit log

Object	Type	Purpose
WMS_AI_API_LOG	table	One row per write-API run: api_id, api_name, method, url, request_body (truncated 4000), http_status, success Y/N, response_text (truncated), instance, source (CHAT / APIS_TAB), invoked_by, created_date. NO procedure or handler needed — the client INSERTs through ai/executewrite.

4 REST endpoints — ORDS module WAREHOUSEMANAGEMENT, prefix /ai

Base: https://g09254cbbf8e7af-graysprod.adb.eu-frankfurt-1.oraclecloudapps.com/ords/WKSP_GRAYSAPP/WAREHOUSEMANAGEMENT/ai

Endpoint	Method	Request	Notes
/getobjectslist	GET	optional ?object= (declared URI param)	Tables/views + columns + comments, honoring WMS_AI_OBJECT_ACL. Builds the model's schema catalog.
/executequery	POST	{"sql":"SELECT ...", "maxRows":200, "appUser":"..."}	SELECT-only gateway (wms_ai_execute_query).
/executewrite	POST	{"sql":"INSERT ...", "appUser":"..."}	Approved DDL/DML AND the API audit-log INSERTs.
/reports/save	POST	{name, category, description, sql, params:[{name,label,type,required,default}]}	
/reports/list	GET	—	All reports (id, name, category, dates).
/reports/get	GET	?id= (URI param)	Report + its params.
/reports/run	POST	{reportId, params:{P_TRIP_DATE:"2026-09-12",...}}	Live re-run.
/reports/delete	POST	{reportId}	
/jobs/create	POST	job JSON — see section 5.2	Returns {success, jobId, firstRun}.
/jobs/list	GET	?fromdate=&todate=&status= (URI params)	Includes next_run from user_scheduler_jobs.
/jobs/get	GET	?id=	Job + last 20 runs with logs.
/jobs/cancel	POST	{jobId}	
/jobs/runnow	POST	{jobId}	

ORDS gotchas encountered (important for the React team): GET query parameters must be declared as handler Parameters (Name/Bind, Source=URI, IN, STRING) or ORDS throws 555 / ORDS-25001. Complex handler bodies are best moved into stored procedures (compile errors then show in user_errors). Use CHR(58) when a PL/SQL literal must contain a colon to keep ORDS bind scanning quiet. APEX_JSON array paths are 1-BASED.

Existing endpoint reused (not new): GET /ords/WKSP_GRAYSAPP/ARMODULE/fusion → {items:[{username, password1}]} = Fusion Basic-auth credentials for live (non-job) Fusion calls.

5 The AI protocol (engine-independent)

5.1 System prompt

A generated markdown document — desktop app: C:\fusion\ai_chat\workspace\CLAUDE.md; React: the API **system** parameter with `cache_control {"type":"ephemeral"}` on the block. Contents: the action protocol below, SQL rules, the compacted schema catalog from `/getobjectslist`, the Fusion API catalog, the write-API catalog summary, and scheduling / email / report / instance rules. Versioned with a marker (FUSION-CATALOG-V10); regenerated when the marker changes, every 24 h, or after a successful DDL.

5.2 Actions the model may return (exactly one JSON object per turn)

```
{ "action":"sql", "sql":"SELECT ...", "reason":"one line" }

{ "action":"fusion", "method":"GET|POST|PATCH|DELETE", "path":"/fscmRestApi/...",
  "body":{"...}, "instance":"PROD|TEST", "reason":"..." }

{ "action":"grid", "markdown":"...", "columns":[...],
  "rows":[{"data":{"...}}...], "actions":[{"id,label,instruction}] }

{ "action":"api_form", "apiId":"trips.create", "values":{"...},
  "note":"markdown shown above the form" }

{ "action":"save_report", "name":"...", "category":"...", "description":"...",
  "sql":"...", "params":[...] }

{ "action":"email", "to":"a@x;b@y", "cc":""," "subject":"...",
  "bodyHtml":"<p>...</p>", "reason":"..." }

{ "action":"db_write", "sql":"CREATE TABLE ... | INSERT ...", "reason":"..." }

{ "action":"schedule_job", "name":"...",
  "scheduleType":"ONCE|RECURRING|REPEAT_UNTIL_DONE",
  "startAt":"YYYY-MM-DD HH24:MI", "intervalMinutes":10,
  "maxRuns":50, "untilDate":"YYYY-MM-DD",
  "instance":"PROD|TEST (omit -> DB default)",
  "completionSql":"SELECT ...",
  "steps":[ {type:"rest", method, url, auth:"none|fusion", body,
    extract:{VAR:"json.path"}},
    {type:"sql", sql} ] }

{ "action":"answer", "markdown":"final answer" }
```

Feedback markers (sent as the next user message): SQL_RESULT:, FUSION_RESULT:, API_RESULT:, DB_WRITE_RESULT:, JOB_RESULT:, EMAIL_RESULT:, REPORT_SAVE_RESULT:, GRID_ACTION: (user ticked rows + clicked a button), and USER_REJECTED for declined approvals.

5.3 Instance context

```
[CURRENT_INSTANCE: TEST]
<actual user message>
```

Every user turn is prefixed by the client with that context line. Model rules: apply it to fusion.instance, schedule_job.instance, and SQL filters on INSTANCE_NAME columns (fixed filter in saved reports). The user's explicit words ("in PROD") override the chip.

6 Engines: CLI vs API — React guidance

	Claude CLI mode	Claude API mode
Billing	Claude subscription (per user)	API key, pay per token
Transport	spawn claude -p --input-format stream-json --output-format stream-json --verbose [--resume <sessionId>]	POST https://api.anthropic.com/v1/messages, headers x-api-key, anthropic-version: 2023-06-01
Context	CLI keeps the session (resume by id)	Client sends system + last 12 turns each call

	Claude CLI mode	Claude API mode
Caching	Automatic	cache_control {"type":"ephemeral"} on the system block (~90% discount on cached reads)
Models	Account default	claude-sonnet-5 (default), claude-haiku-4-5, claude-opus-5
In React	NOT available in a browser — only viable via a backend that shells out	Direct fit, BUT never ship the key to the browser: proxy /v1/messages through your backend (or an ORDS/APEX proxy) and keep the key server-side

The WebView app stores the engine choice per PC (localStorage aiEngineSettings); the protocol and system prompt are identical either way.

7 Feature specifications

7.1 Chatbot page (3 panels)

Left: chat history list + Objects (schema browser from /getobjectslist) + Reports shortcuts. **Middle:** conversation with round chips (each SQL/Fusion round: statement, rowcount, ms, expandable result). **Right:** results panel with Excel / CSV / PDF-print / Copy-SQL export of the last grid.

Slash commands (typing / filters): /f* fusion templates, /a* apex templates, /trip-manual-lines, /order-lines-grid, /create-trip, /add-orders, /email, /email-settings, /save-report, /reports, /schedule, /jobs, /apis, /engine, /help, /new.

7.2 Approval model (uniform)

Fusion writes, emails, DB writes and job creation return requiresApproval + pending* → the UI renders a card (a summary of exactly what will run) with Approve / Reject → the decision endpoint resumes the model with the result marker. Write-API forms are approved differently: the **form itself plus a confirm dialog** (method, final URL, body, instance) — see 7.6.

7.3 Saved reports

"Save as report" button on answers (or /save-report) → name/category dialog → the model emits save_report with the SQL and typed parameters. Saved Reports tab: left list (category-grouped, API-inspector link showing the exact run payload), right preview grid + parameter prompt dialog + exports. Re-runs are always live (/reports/run).

7.4 Scheduled jobs (DBMS_SCHEDULER)

Chat designs the job (schedule_job) → purple approval card (schedule line, step plan, completion SQL, "runs inside the database" notice) → /jobs/create stamps created_by + machine. Scheduled Jobs tab: date/status/name filters, job detail (step plan, audit chips, run history with expandable logs), Run Now / Cancel, 30 s auto-refresh. Jobs are instance-aware (section 3.4) and keep running when the app is closed.

7.5 Emailing

email action → approval card with recipients + rendered HTML body → the desktop app sends via smtp.office365.com:587 STARTTLS with the user-entered account (App Password when MFA is on). **React note:** browsers cannot speak SMTP — send through your backend, or use Microsoft Graph sendMail with OAuth (the cleaner web-app path).

7.6 Write-API catalog + interactive forms

- **Catalog** (aianalysis/api-catalog.js, ~18 entries): id, name, module, method, URL template ({path} params), field specs (text|number|date|textarea|json|rows, in: body|query|path, required, defaults), instanceIn = where the current instance is injected (p_instance_name in body or query). Flagship entries: trips.create, trips.addorders, trip.updatetrip, trip.assignpicker, pick wave / pick release, order-line cancellations, S2V/store ops (raw-JSON forms).
- **APIs tab:** searchable module-grouped list → detail (desc, method + URL, instance note) → auto-generated form → Run... → confirm dialog → execute → response viewer.
- **api_form chat flow:** the model ends its turn with api_form (apild + prefilled values; instance fields never included — the client injects them). The client renders the same form in the chat; after confirm + run it starts a new turn API_RESULT: {apild, success, response} so the model confirms and REMEMBERS returned ids (e.g. the new trip_id) as defaults for follow-ups.

- **Add-orders validation flow:** user pastes order numbers → the model FIRST runs sql validating them against pending shipment lines for the current instance (exists + not already on a trip) → then `api_form` for `trips.addorders` with valid orders as pre-ticked checkbox rows (invalid ones listed with reasons in the note); `trip_id` defaults to the trip created/discussed in the conversation.
- **Audit:** every run (chat or tab) INSERTs into `WMS_AI_API_LOG` through `/ai/executewrite` — no dedicated logging endpoint.

7.7 Instance selector

Topbar chip PROD (green) / TEST (amber); default = login instance (`loggedInInstance` / `fusionInstance`), persisted (`aiInstance`); the toggle announces the switch in-chat. Drives: the `[CURRENT_INSTANCE]` context line, catalog `instanceIn` injection, Fusion base URL choice (live calls: `efmh` vs `efmh-test`), and job defaults.

7.8 Fusion integration

Live calls use Basic auth with credentials from `ARMODULE/fusion`; the catalog includes `shipmentLines` GET, `salesOrdersForOrderHub` PATCH (line cancel), `pickTransactions` / `shipmentTransactionRequests` / `shippingTransactions` POST, `inventoryStagedTransactions` GET/DELETE. The keyword "fusion" (or `/f*` commands) routes the model to Fusion instead of SQL. Writes always pause for approval. Scheduled-job REST steps instead use credentials/instance from `WMS_AI_SETTINGS` (section 3.4).

8 Guardrails summary

- **SQL gateway:** single SELECT/WITH only, keyword ban, row cap, ACL-filtered catalog, full query log.
- **DB writes:** verb whitelist, dangerous-package/statement ban, always user-approved.
- **Jobs:** min 2-minute interval, ≤ 200 runs, ≤ 30 -day lifetime, REST host whitelist, self-terminating, full run logs, `created_by/machine` audit.
- **Write APIs:** catalog-only (the model cannot invent URLs), client-injected instance, mandatory confirm dialog, `WMS_AI_API_LOG` audit.
- **Approvals:** nothing outbound (Fusion write, email, DDL/DML, job, API) executes without an explicit user click.

9 React porting checklist

Reuse unchanged (zero backend work): all tables (section 3), all `/ai/*` endpoints (section 4), the action protocol + system-prompt content (section 5), the API catalog data (section 7.6 — port `api-catalog.js` as a TS module), and the guardrails.

- **Reimplement in React:** chat UI (3 panels, cards, forms, grids, tabs), instance chip, slash palette, exports (e.g. `exceljs` + `print CSS`), state (conversation array instead of a CLI session).
- **Needs a server component:** Claude API proxy (protect the key; add the `cache_control` system block there), email sending (Graph API or an SMTP relay), and optionally serving the system prompt / catalog centrally so all clients update at once.
- **Not portable:** Claude CLI mode, `C:\fusion` workspace files, `WebView2` IPC (replace `sendMessageToCSharp` with `fetch` to ORDS / your proxy).

Repository copy: `docs/AI_ANALYSIS_RD.md` on branch `claude/new-session-j44czz` · Gray's WMS v10.0.0